

GRC ENGINEER / PROJECT PRESENTATION

Implementing Continuous Control Monitoring with AWS, ServiceNow IRM, and Python

Jonathan Steward August 19, 2026

The Problem

WHAT IS WRONG?

The Enterprise ServiceNow GRC (IRM) platform is disconnected from the most critical environment to business operations



Increased risk of security incidents and non-compliance due to lack of visibility

Assessments are manual and point-in-time



Risk decisions are being made on a control effectiveness state that may be months out of date

Teams are manually collecting audit evidence



The amount of time and budget spent on audit efforts increases with every cycle and pulls control owners and engineers off delivery

Control deficiencies are found after production deployment and remediations are not prioritized correctly



Vulnerable changes are being pushed to production which increases our external attack surface risk

WHY SHOULD WE FIX IT?

The Solution

A continuous AWS security control monitoring pipeline using AWS Config rules, Python, Claude, GitHub Actions, and ServiceNow IRM to enable continuous compliance, drift prevention, and drift detection.

The pipeline enables automated evidence collection, provides AI-generated remediation guidance, enables FAIR-informed risk quantification, and daily reporting.

Prevention

Non-compliant infrastructure is blocked at the pull request before it can be pushed to production

`Terraform → Conftest policy gate → required check on every PR`

Detection

Control deficiencies are caught daily, centralized in one platform, prioritized based on risk, and sent to the risk and control owners.

`AWS Config → ServiceNow IRM → quantified risk → daily executive report`

03 First Iteration

FIRST ITERATION – STATUS SYNC ONLY

01 **Run Daily GitHub Actions Job**



02 **Call AWS Config API to Get Compliance Status of Key AWS Config Rules**



03 **Push Evidence and Compliance Status to ServiceNow IRM Mapped CIS Control Record**

CHALLENGE

This design acted as a good baseline to ensure the GRC team had a centralized place to review data, but it didn't improve risk prioritization or decision making for risk owners, leadership, or other teams.

04 Current Architecture

DAILY PIPELINE — RUNS AT 8 AM CST

01 Run Daily GitHub Actions Job



02 Call AWS Config API to Get Compliance Status of Key AWS Config Rules



03 Push Evidence and Compliance Status to ServiceNow IRM Mapped CIS Control Record



ADDED THIS CYCLE

04 Send Non-Compliant Controls to Claude API and Receive Remediation Guidance



05 Quantify Risk using FAIR-informed Risk Framework and Document in ServiceNow



06 Create Daily PDF Report Summary for Non-Compliant Controls



07 Send Report to Slack and Risk/Control Owner Email

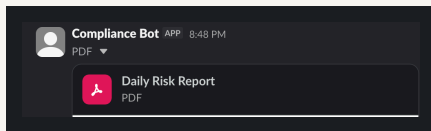
PARALLEL TRACK — PRE-MERGE

Policy-as-code gate

Terraform plan evaluated against OPA Rego policies via Conftest in the CI/CD pipeline, required branch-protection check — non-compliant changes blocked before merge

05 Report Example

SLACK DELIVERY



Uploaded as a PDF by the pipeline's Slack app

\$720,000/yr

Annual exposure on this finding, carried on risk record RK0020001

DAILY RISK REPORT — ONE CONTROL ROW

Control	Rating	Non-Compliance Summary
iam-password-policy 5.2 Use Unique Passwords	Medium High	The account-wide IAM password policy for account [AWS ACCOUNT REDACTED] fails to meet one or more required parameters (minimum length of 14, reuse prevention of 5, max age of 90 days, and mandatory lowercase/uppercase/number/symbol complexity). AWS Config returned no specific annotation, suggesting the policy is either missing entirely or deficient in at least one required setting. Because this is a single account-wide configuration, any shortfall in these thresholds causes the entire account to be marked NON_COMPLIANT, leaving the full inherent risk exposure unmitigated.

Extending the GRC Platform

Built code inside ServiceNow and against its REST Table API – GET / PATCH / POST

Control status

Custom JavaScript in ServiceNow updates each CIS control with its mapped AWS config rule compliance status

Risk quantification

Python enables FAIR-informed quantitative risk assessment and risk statements, then stores the record in ServiceNow's native risk record format.

Reporting

Python builds the compliance report and sends it to Slack and email

IMPORTANT TO NOTE

ServiceNow stays the system of record. Auditors, GRC, engineering and IT can all work from one place.

What Shaped the Design

- 01 GRC lead has a growing workload across audit, compliance, and risk assessment
- 02 Risk owners, leadership, and security need prioritized remediation efforts and a common risk language
- 03 Reports were being delivered to email and were not being read
- 04 Configuring and customizing the ServiceNow IRM platform was a heavy lift for limited value
- 05 Remediating non-compliant infrastructure after deployment costs more than blocking it pre-merge

Decisions and Trade-offs

CONSIDERED	CHOSEN	WHY
Native ServiceNow-to-AWS integration (Flow Designer/Scripted REST)	Python orchestrator, ServiceNow as system of record	Python is not locked to one runtime and already runs locally and in GitHub Actions, portable to AWS Lambda with no rewrite
Multi-agent for remediation guidance	Call to Claude API for remediation guidance as needed	Call to the LLM for guidance is exactly what we need. We do not need an agent to take action.
Slack Webhook to read JSON straight to Slack group	Bot Token app with PDF file upload	As the number of resources grows, reading the output directly to Slack would be impossible to read

Measuring Success and Key Takeaways

Success criteria

Control deficiencies are mapped to risks and prioritized for improved decision making

Relevant owners and teams receive timely and actionable information related to control deficiencies

Evidence is tracked in a centralized platform for ease of auditability

What I would do differently

Take time to consider all process stakeholders before building anything

Extend the platform if it saves time or enables more effective processes

Be ok with an 80% solution because continuous improvement is important

10

Questions?

11

Appendix

APPENDIX A AWS Config Rule to CIS v8 Control Mapping

Seven AWS Config rules mapped to CIS v8 controls

Control mapping table

AWS Config Rule	CIS v8 Control Number	CIS Control Name
root-account-mfa-enabled	6.5	Require MFA for Administrative Access
iam-password-policy	5.2	Use Unique Passwords
cloudtrail-all-read-s3-data-event-check	3.14	Log Sensitive Data Access
	8.2	Collect Audit Logs
s3-account-level-public-access-blocks-periodic	12.2	Establish and Maintain a Secure Network Architecture
s3-bucket-public-read-prohibited	12.2	Establish and Maintain a Secure Network Architecture
s3-bucket-public-write-prohibited	12.2	Establish and Maintain a Secure Network Architecture
subnet-auto-assign-public-ip-disabled	12.2	Establish and Maintain a Secure Network Architecture

APPENDIX B **AWS Config Rule Compliance Status in ServiceNow**

Custom table holding the latest status per rule

ServiceNow – u_aws_config_control_mapping

AWS Config Rule Name ▾	Status	Updated
subnet-auto-assign-public-ip-disabled	NON_COMPLIANT	2026-08-16 18:11:21
root-account-mfa-enabled	COMPLIANT	2026-08-17 18:20:48
iam-password-policy	NON_COMPLIANT	2026-08-17 18:20:48
cloudtrail-all-read-s3-data-event-check	NON_COMPLIANT	2026-08-17 18:20:49

APPENDIX C Evidence Records — One Row Per Resource Per Run

Append-only child table; account identifiers redacted

ServiceNow — evidence child table

☐		Created in ServiceNow ▾	Captured In External Platform	Compliance Status	Control Mapping	Resource ID	Resource Type	Standard Summary
		2026-08-18 18:48:22	2025-12-08 11:08:47	NON_COMPLIANT	subnet-auto-assign-public-ip-disabled		AWS::EC2::Subnet	This AWS Config rule enforces the securi...
		2026-08-18 18:48:21	2025-12-08 11:08:50	NON_COMPLIANT	subnet-auto-assign-public-ip-disabled		AWS::EC2::Subnet	This AWS Config rule enforces the securi...
		2026-08-18 18:48:21	2026-08-18 13:32:59	NON_COMPLIANT	cloudtrail-all-read-s3-data-event-check		AWS::::Account	This rule enforces a common security bes...
		2026-08-18 18:48:21	2026-08-18 13:32:59	NON_COMPLIANT	iam-password-policy		AWS::::Account	This rule enforces a baseline password s...
		2026-08-18 06:37:16	2026-08-17 18:31:02	NON_COMPLIANT	iam-password-policy		AWS::::Account	This rule enforces a baseline password s...

APPENDIX D Control Status Update Script

GlideRecord script that sets the mapped CIS control status from the AWS Config result

ServiceNow – compliance indicator script

```
1 // AWS Config rule associated with this indicator
2 var ruleName = "root-account-mfa-enabled";
3
4 // Query AWS Config results table
5 var gr = new GlideRecord('u_aws_config_control_mapping');
6 gr.addQuery('u_aws_config_rule_name', ruleName);
7 gr.setLimit(1);
8 gr.query();
9
10 // Get the specific ServiceNow control record
11 var gr2 = new GlideRecord('sn_compliance_control');
12
13 if (gr2.get('522e4a7c837e071085a5c310feaad3ec')) {
14     if (gr.next()) {
15         var status = gr.getValue('u_status');
16
17         if (status == 'COMPLIANT') {
18             result.value = 100;
19             result.passed = true;
20
21             gr2.setValue('status', 'compliant');
22
23         } else if (status == 'NON_COMPLIANT') {
24             result.value = 0;
25             result.passed = false;
```

```
30
31     gr2.setValue('status', 'non_compliant');
32
33 } else {
34     result.value = 0;
35     result.passed = false;
36
37     gr2.setValue('status', 'non_compliant');
38 }
39
40 // Attach AWS Config record as supporting evidence
41 result.supportingDataIds = [
42     gr.getUniqueValue()
43 ];
44
45 } else {
46     // No AWS Config evidence found
47     result.value = 0;
48     result.passed = false;
49
50     gr2.setValue('status', 'non_compliant');
51 }
52
53 // Save the control status change
54 gr2.update();
55
56 }
57 }
```

APPENDIX E Mapped CIS Control Record

Control 5.2 with status driven by the pipeline

ServiceNow — sn_compliance_control

* Name 5.2 Use Unique Passwords

Number CTRL0020005

* Entity AWS  

Status Non Compliant

State Draft

Exempt ☒

APPENDIX G

FAIR-informed exposure range and gap narrative; account identifier redacted

ServiceNow – risk assessment

* Name	AWS Config: iam-password-policy (5.2 Use Unique Passwords) - risk assessment		
Description	Inherent exposure (if this control provided no protection at all): \$287,945-\$1,271,549 (90% confidence range), median \$667,313. Control is currently non-compliant, so no risk reduction is being realized - residual exposure equals inherent. Based on 10,000 Monte Carlo samples of illustrative threat-frequency, control-effectiveness, and loss-magnitude ranges, reasoned from published industry breach-cost benchmarks - not this organization's own incident history. Current gap: The account-level IAM password policy for account [REDACTED] does not fully satisfy the configured requirements (min length 14, reuse prevention of 5, max age 90 days, and mandatory lowercase/uppercase/number/symbol complexity). AWS Config did not provide a specific annotation, indicating either the password policy is missing entirely or one or more of its settings (length, age, reuse, or character composition) fall short of the specified thresholds. Since IAM account password policy is a single account-wide setting, any deviation from the required parameters causes the entire account to be marked NON_COMPLIANT.		
Key Statement	Credential-based attacks against IAM us	Entity	AWS
Category	None		

APPENDIX H Daily Pipeline Workflow

Scheduled GitHub Actions job — secrets, Python setup, main.py

[.github/workflows](#) — daily sync

```
jobs:
  sync:
    runs-on: ubuntu-latest

    env:
      SN_I: ${ secrets.SN_I }
      SN_T: ${ secrets.SN_T }
      SN_U: ${ secrets.SN_U }
      SN_P: ${ secrets.SN_P }
      AWS_ACCESS_KEY_ID: ${ secrets.AWS_ACCESS_KEY_ID }
      AWS_SECRET_ACCESS_KEY: ${ secrets.AWS_SECRET_ACCESS_KEY }
      AWS_DEFAULT_REGION: ${ secrets.AWS_DEFAULT_REGION }
      ANTHROPIC_API_KEY: ${ secrets.ANTHROPIC_API_KEY }
      SLACK_BOT_TOKEN: ${ secrets.SLACK_BOT_TOKEN }
      SLACK_CHANNEL_ID: ${ secrets.SLACK_CHANNEL_ID }
      EMAIL_FROM: ${ secrets.EMAIL_FROM }
      EMAIL_TO: ${ secrets.EMAIL_TO }
      SMTP_SERVER: ${ secrets.SMTP_SERVER }
      SMTP_PORT: ${ secrets.SMTP_PORT }
      EMAIL_APP_PASSWORD: ${ secrets.EMAIL_APP_PASSWORD }

    steps:
      - name: Checkout code
        uses: actions/checkout@11d5960a326750d5838078e36cf38b85af677262 # v4

      - name: Set up Python
        uses: actions/setup-python@a26af69be951a213d495a4c3e4e4022e16d87065 # v5
        with:
          python-version: '3.10'

      - name: Install dependencies
        run:
          pip install -r requirements.txt

      - name: Run sync script
        run:
          python main.py
```

APPENDIX I Policy-as-Code Gate Workflow

Terraform plan converted to JSON and evaluated by Conftest against OPA Rego policies

`.github/workflows - pull request gate`

```
EOF

- name: Terraform init
  working-directory: ${ matrix.dir }
  run: terraform init -input=false

- name: Terraform plan
  working-directory: ${ matrix.dir }
  run: terraform plan -input=false -out=tfplan

- name: Convert plan to JSON
  working-directory: ${ matrix.dir }
  run: terraform show -json tfplan > plan.json

- name: Install Conftest
  run: |
    curl -sSLo conftest.tar.gz https://github.com/open-policy-agent/conftest/releases/download/v0.69.0/conftest_0.69.0_Linux_x86_64.tar.gz
    echo "96fc2fbf11f0afde51256647127e6f00a64ce839a4d9a0a1aef2426c0e6f4b3f  conftest.tar.gz" | sha256sum -c -
    tar -xzf conftest.tar.gz conftest
    chmod +x conftest
    sudo mv conftest /usr/local/bin/conftest

- name: Run Conftest
  working-directory: ${ matrix.dir }
  run: |
    set -o pipefail
    conftest test plan.json --policy ../../policy --no-color | tee conftest-output.txt
    CONFTTEST_EXIT=$?
    {
      echo "### Compliance check: `${ matrix.dir }`\"
      echo \"\"
      cat conftest-output.txt
      echo \"\"
    } >> "$GITHUB_STEP_SUMMARY"
    exit $CONFTTEST_EXIT
```